

Nota: O exame deve ser respondido em 3 folhas de prova separadas:

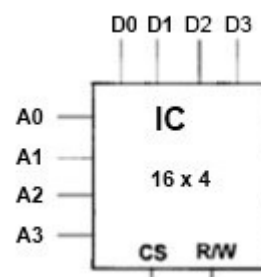
- 1 Folha para responder às três perguntas Teóricas
- 1 Folha para responder à 1ª pergunta da Prática
- 1 Folha para responder à 2ª pergunta da Prática

Duração Total Exame (T + P) : 2h:00m

19 de Junho de 2026

Parte Teórica

- 1** Pretende-se desenvolver um sistema de aquisição de dados com uma pequena memória que tenha capacidade para armazenar 50 Bytes. A memória deve ser construída com recurso a Circuitos Integrados de memória RAM representados na figura, onde A₃, A₂, A₁, A₀ representam linhas de endereço, D₃, D₂, D₁, D₀ representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Selection*.



Faça um esboço da memória pretendida, indicando as linhas de dados, endereços, CS e R/W da memória resultante, garantindo que cada endereço de memória guarda a informação correspondente a 1 Byte. O número de integrados usados deve ser o mínimo possível. **(2 Val)**

- 2** Considere uma máquina com uma *cache* com mapeamento direto, blocos de 1 Byte e com 7 bits de *tag*. Esta máquina tem uma memória RAM com 2KB de capacidade.

- a) Qual a função dos *valid bits*? Justifique. **(0,5 Val.)**
b) Qual a capacidade total desta *cache*, contando com os bits da *tag* mais os *valid bits*? Deve apresentar todos os cálculos que permitem determinar o valor pedido. **(2 Val.)**

- 3** A tecnologia de *pipeline* é uma técnica amplamente utilizada em processadores para melhorar o desempenho da execução de instruções.

- a) Explique o que é *pipeline* no contexto de processadores. **(0,5 Val.)**
b) Apresente um exemplo de como uma sequência de instruções em Assembly 8086 poderia se beneficiar de uma execução com *pipeline*. **(1 Val.)**
c) Cite e explique dois tipos de problemas (limitações) que podem ocorrer em uma *pipeline* durante a execução de um programa Assembly. E também que soluções foram implementadas para contornar estas limitações. **(1 Val.)**

Parte Prática

1. Considere um **vetor** de números inteiros positivos (cada valor pode ser no máximo 255), que podem estar desordenados. O vetor termina com o valor 0, que não deve ser considerado para efeitos de cálculo.

Pretende-se um programa em linguagem Assembly 8086 para determinar a diferença entre o maior e o menor dos valores existentes no **vetor**.

Exemplo:

(2,5 Val.)

vetor = 3; 60; 34; 5; 2; 7; 34; 0

diff = 60 – 2 = 58

2. Considere o jogo Tic-Tac-Toe, também conhecido por Jogo do Galo. Foi desenvolvido um programa simples em Assembly que, utilizando o modo de cor do 8086, permite a duas pessoas, com os caracteres minúsculos 'o' e 'x' representando os círculos e cruzeiros do jogo original, jogarem uma partida até à vitória de uma delas. O programa encontra-se quase totalmente desenvolvido, permitindo já aos jogadores definirem o seu nome, as cores que querem utilizar e jogar alternadamente até à vitória.

Para terminar este programa **deve criar um procedimento completo chamado INDICA VENCEDOR que deve sublinhar a linha da vitória com as cores do vencedor, colocá-la a piscar e alterar os caracteres da linha vencedora passando de minúsculas para maiúsculas.**

(2,5 Val.)

Deve considerar que:

- Estão definidas as seguintes variáveis:

```
DATA_HERE      SEGMENT
    LINHAVENCEDORA  db 3 dup (3 dup (??))
    VENCEDOR        db ?
    ...
DATA_HERE ENDS
```

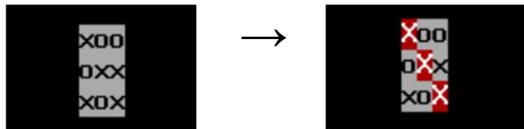
- O tabuleiro é constituído apenas por uma **matriz 3x3** em que cada elemento corresponde às 9 casas de jogo localizado no centro do ecrã (**o canto superior esquerdo do tabuleiro na 11ª linha e 39ª coluna do ecrã**);
- A linha da vitória está definida na variável **LINHAVENCEDORA**, em que os elementos a 1 correspondem às posições da linha vencedora e os restantes estão a 0.
- De forma a facilitar, considere que sempre que o procedimento é chamado é porque já existe um **vencedor que está definido na variável VENCEDOR**, em que o valor é 1 no caso do primeiro jogador ser o vencedor e 2 no caso da vitória ser do segundo. As cores de cada jogador estão definidas **na primeira e última linha do ecrã** respetivamente (logo a partir da primeira célula);
- Garantir que o procedimento não afeta o resto do programa quando é chamado.

Considere os exemplos das partidas dos seguintes jogadores, conforme as variáveis VENCEDOR e LINHAVENCEDORA:

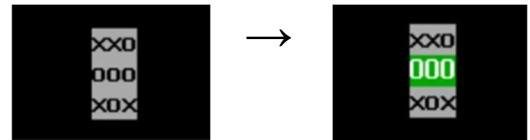
 Jogador 1: FD

 Jogador 2: HB

VENCEDOR 2
LINHAVENCEDORA: 1,0,0,0,1,0,0,0,1



VENCEDOR 1
LINHAVENCEDORA: 0,0,0,1,1,1,0,0,0



Apoio

Composição do BYTE de atributos de célula no modo cor

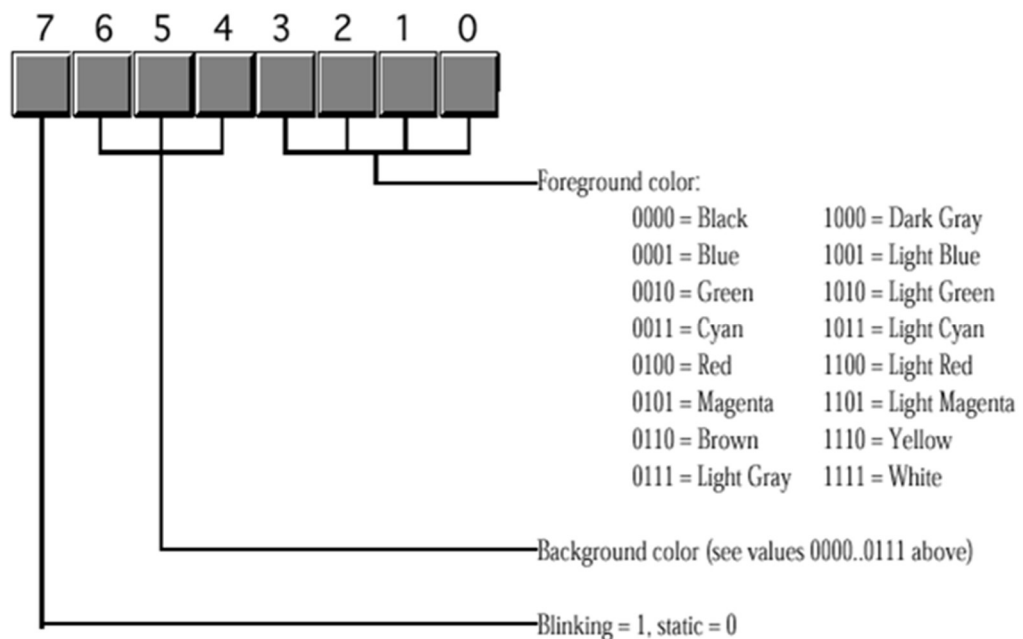


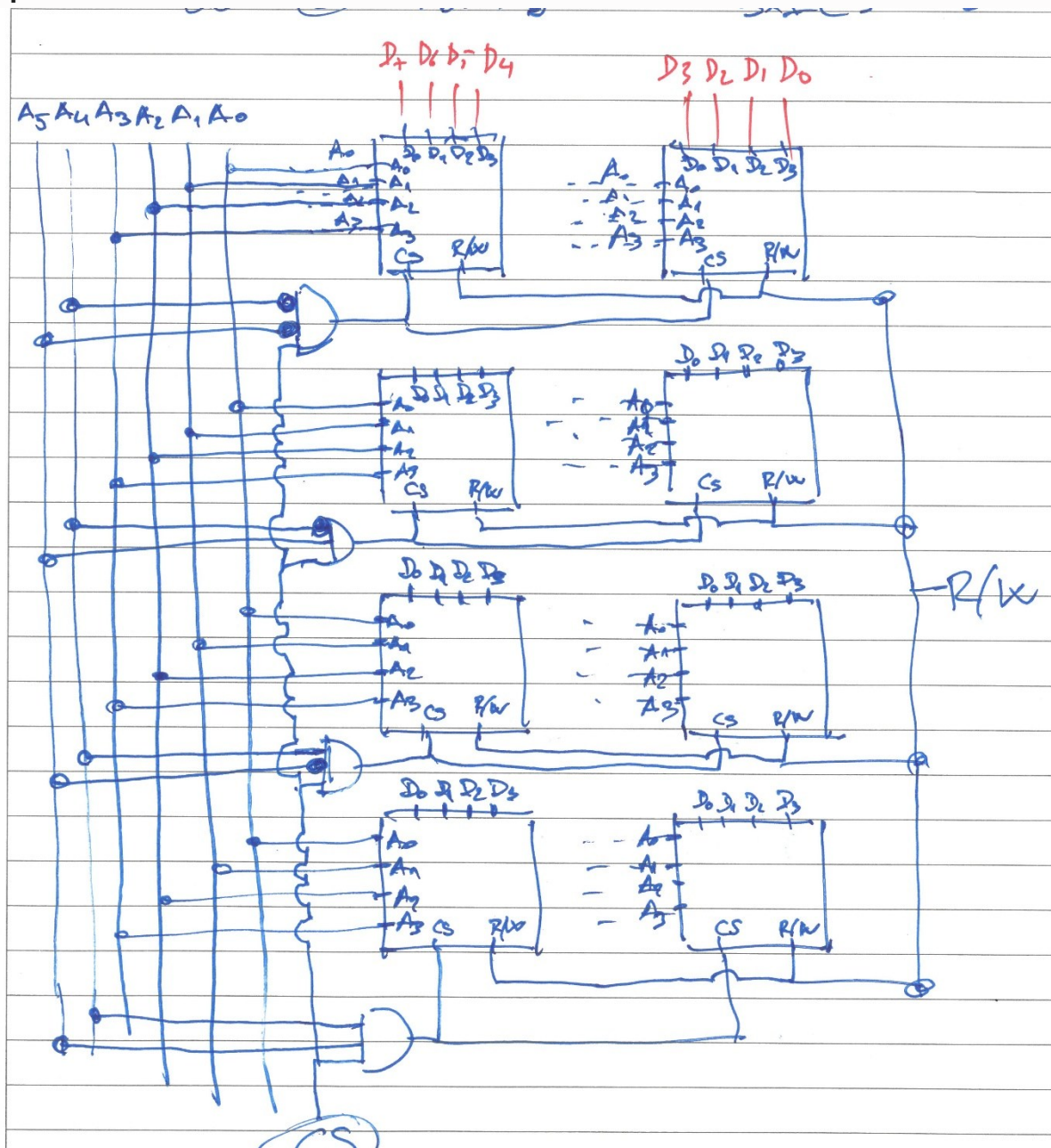
Tabela ASCII

Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
0	0	NUL	32	20	Space	64	40	@	96	60	`
1	1	SOH	33	21	!	65	41	A	97	61	a
2	2	STX	34	22	"	66	42	B	98	62	b
3	3	ETX	35	23	#	67	43	C	99	63	c
4	4	EOT	36	24	\$	68	44	D	100	64	d
5	5	ENQ	37	25	%	69	45	E	101	65	e
6	6	ACK	38	26	&	70	46	F	102	66	f
7	7	BEL	39	27	'	71	47	G	103	67	g
8	8	BS	40	28	(	72	48	H	104	68	h
9	9	HT	41	29	)	73	49	I	105	69	i
10	A	LF	42	2A	*	74	4A	J	106	6A	j
11	B	VT	43	2B	+	75	4B	K	107	6B	k
12	C	FF	44	2C	,	76	4C	L	108	6C	l
13	D	CR	45	2D	-	77	4D	M	109	6D	m
14	E	SO	46	2E	.	78	4E	N	110	6E	n
15	F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

```

.8086
.model small
.stack 2048
DATA_HERE SEGMENT
...
DATA_HERE ENDS
CODE_HERE SEGMENT
    ASSUME CS:CODE_HERE, DS:DATA_HERE
START:mov ax, DATA_HERE
    mov ds, ax
...
    mov ah, 4ch
    int 21h
CODE_HERE ENDS
END START

```



T2

a) Qual a função dos valid bits? Justifique.

O valid bit (bit de validade) indica se a informação presente numa determinada linha da memória cache é válida ou lixo/inválida (por exemplo, após o arranque do sistema ou um reset). Se o bit estiver a 1, a cache contém dados corretos sincronizados com a RAM; se estiver a 0, qualquer acesso a essa linha resultará obrigatoriamente num cache miss, forçando a leitura da memória principal.

b) Capacidade total da cache (incluindo bits de tag e valid bits)

1. Determinar a estrutura do endereço de memória:

Endereco (11 bits)=Tag (7 bits)+Índice (X bits)+Offset (0 bits)

$$11 = 7 + X + 0 \Rightarrow X = 4 \text{ bits para o Índice}$$

2. Número de linhas da cache:

$$\text{Linhas} = 2^{\text{Índice}} = 2^4 = 16 \text{ linhas}$$

3. Estrutura de cada linha da cache:

- Bits de Dados = 1 Byte=8 bits
- Bits de Tag = 7 bits
- Bit de Validade = 1 bit
- **Total por linha** = 8+7+1=16 bits

4. Capacidade Total da Cache:

$$\text{Capacidade Total} = 16 \text{ linhas} \times 16 \text{ bits/linha} = 256 \text{ bits} = \mathbf{32 \text{ Bytes}}$$

T2

a) Explique o que é pipeline no contexto de processadores.

O pipeline é uma técnica de implementação de hardware que permite a **execução sobreposta e simultânea de múltiplas instruções**. Semelhante a uma linha de montagem industrial, uma instrução é dividida em várias etapas independentes (ex: Busca, Decodificação, Execução, Acesso à Memória, Escrita). Enquanto uma instrução está na fase de execução, a instrução seguinte já está a ser decodificada, aumentando o rendimento (throughput) do processador.

b) Exemplo em Assembly 8086 beneficiado por pipeline.

Exemplo em Assembly 8086 beneficiado por pipeline.

Instruções sequenciais e independentes tiram o máximo partido do pipeline:

```
MOV AX, 10      ; Instrução 1: Carrega 10 em AX
ADD BX, CX      ; Instrução 2: Totalmente independente da Instrução 1
MOV DX, [SI]    ; Instrução 3: Totalmente independente das anteriores
```

Como beneficia: Enquanto `MOV AX, 10` está a ser executada no estágio de execução, `ADD BX, CX` está a ser decodificada, e `MOV DX, [SI]` está a ser procurada na memória. Não há paragens (stalls).

c) Dois tipos de problemas (limitações/hazards) e soluções.

1. Conflitos de Dados (Data Hazards):

- Problema: Ocorre quando uma instrução depende do resultado de uma instrução anterior que ainda não terminou a sua execução. Exemplo:

```
ADD AX, BX      ; O resultado final só é escrito em AX no último estágio
SUB CX, AX      ; Precisa do novo valor de AX imediatamente
```

- Solução: **Data Forwarding / Bypassing** (encaminhamento de hardware que passa o resultado diretamente da unidade de execução para a entrada da instrução seguinte sem esperar pela escrita no registo) ou inserção de bolhas (stalls/NOPs).

o

2. Conflitos de Controlo (Control Hazards):

- Problema: Provocado por instruções de salto condicional ou incondicional (`JMP`, `JE`, `JNZ`). O processador não sabe qual será a próxima instrução a entrar no pipeline até que a condição do salto seja avaliada.
- Solução: **Predição de Saltos (Branch Prediction)** (o hardware tenta adivinhar se o salto será tomado ou não com base no histórico) ou **Delayed Branch**.

P1

```
.8086
.model small
.stack 2048

dseg segment para public 'data'
    MAX db ?
    MIN db ?
    DIF db ?
    VEC db 3,60,34,5,2,7,34,0
dseg ends

cseg segment para public 'code'
    assume cs:cseg, ds:dseg
main proc
    mov ax, dseg
    mov ds, ax

    xor si,si
    mov al,VEC[si]
    mov MAX,al
    mov MIN,al
    inc si
volta:
    mov al,VEC[si]
    cmp al,0
    je fim
    cmp al,MAX
    jb cont1
    mov MAX,al
cont1:
    cmp al,MIN
    ja cont2
    mov MIN,al
cont2:
    inc si
    jmp volta

fim:
    mov al,MAX
    sub al,MIN
    mov DIF,al

    mov al, 0
    mov ah, 4ch
    int 21h
main endp
cseg ends

end main
```


INDICA VENCEDOR PROC ;declaração do procedimento

;GUARDAR CONTEXTO

PUSHF

PUSH AX

PUSH BX

PUSH SI

PUSH DI

PUSH ES

;apontar ES para memória de vídeo (modo cor)

MOV AX, 0B800h

MOV ES, AX

;obter formato do jogador vencedor e preparar byte de formato de células

MOV AL, vencedor

CMP AL, 1

JNE venceJ2

MOV AH, ES:[1]

JMP flipBlink

venceJ2:

MOV AH, ES:[24*160+1]

flipBlink:

OR AH, 10000000b ;actualizar formato (flip MSb)

;calcular posição inicial do tabuleiro ([10,38]) e preparar ciclo

MOV SI, 10*160+38*2 ;alternativamente poderiam fazê-lo com MULs e ADDs, tendo cuidado com formato das instruções, tamanhos e sobreposições de registos

LEA DI, linhaVencedora

XOR BX, BX

cicloLinha:

CMP BH, 3

JE fimCicloLinha

cicloColuna:

CMP BL, 3

JE proxLinha

CMP byte ptr [DI], 1

JNE proxCol

MOV AL, byte ptr ES:[SI]

SUB AL, 20h ;actualizar caracter

MOV ES:[SI], AX ;substituir célula (caracter+atributos)

proxCol:

INC BL

ADD SI, 2

INC DI

JMP cicloColuna

proxLinha:

XOR BL, BL

INC BH

ADD SI, 154 ;(+linha -deslocamento colunas tabuleiro)

JMP cicloLinha

fimCicloLinha: ;RETORNAR CONTEXTO

POP ES

POP DI

POP SI

POP BX

POP AX

POPF

RET ;retorno do procedimento

INDICA VENCEDOR ENDP ;fim do procedimento

;202606.hb